

ISIS 2503 - Arquitectura y Diseño de Software

Tema
Tácticas de seguridad
Auth0 y JWT

Agenda

- Objetivos
- Tácticas de arquitectura de software de seguridad
- Implementación de tácticas con Auth0 y JSON Web Tokens

Objetivos

Al finalizar este módulo el estudiante estará en capacidad de:

- Seleccionar tácticas de arquitectura de software para mitigar amenazas de seguridad
- Implementar tácticas de seguridad usando tecnologías concretas como Auth0 y JWT

OWASP (Open Web Application Security Project)

- OWASP es un patrón que identifica amenazas de seguridad que deben ser tenidas en cuenta por el arquitecto
- A continuación vamos a ver qué tácticas de seguridad mitigan algunas amenazas mencionadas en OWASP
 - Las tácticas aparecen en azul
 - Las amenazas aparecen resaltadas con rojo

Tácticas contra amenazas (1/2)

Hashing

Encriptación simétrica o asimétrica

Sensitive data exposure

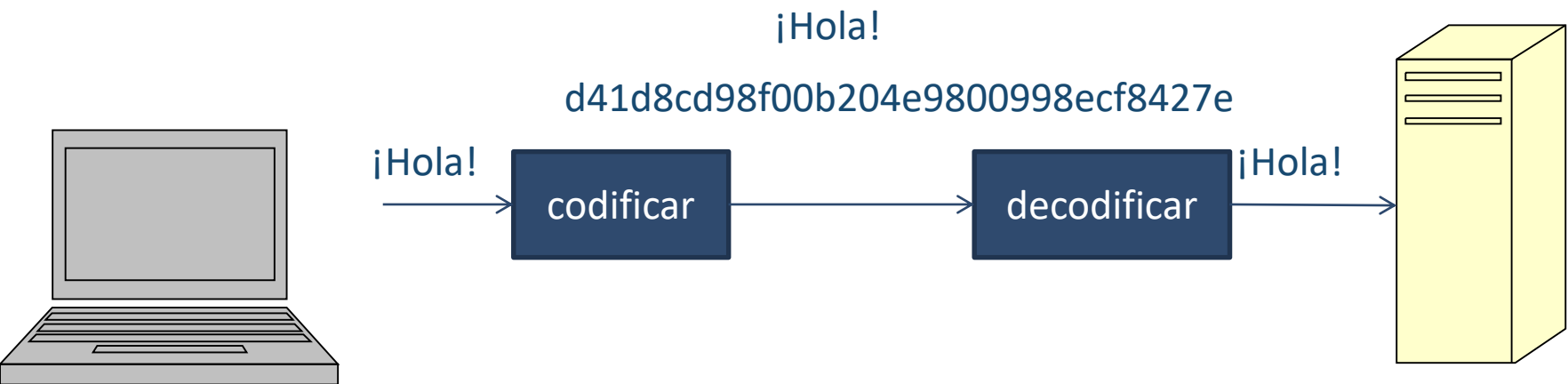
Certificados digitales (signature)

Broken access control

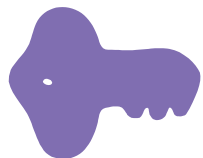
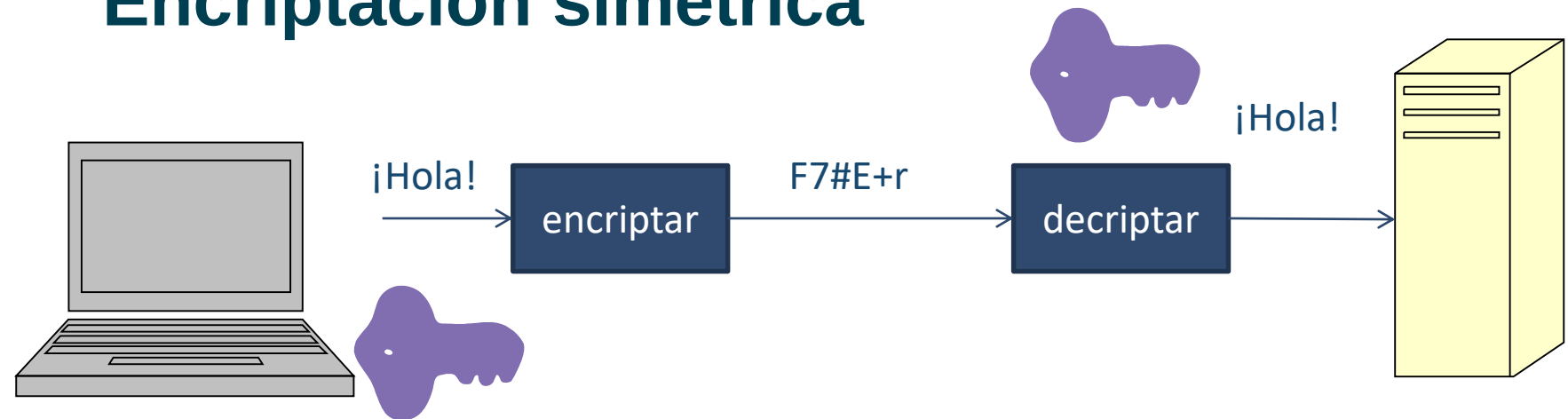


Mitigar
amenazas

Hashing



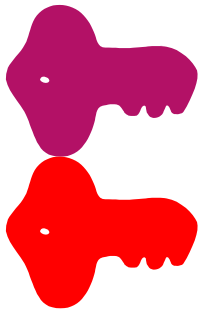
Encriptación simétrica



Llave única

Llave única se genera y se envía cada cierto tiempo

Encriptación asimétrica



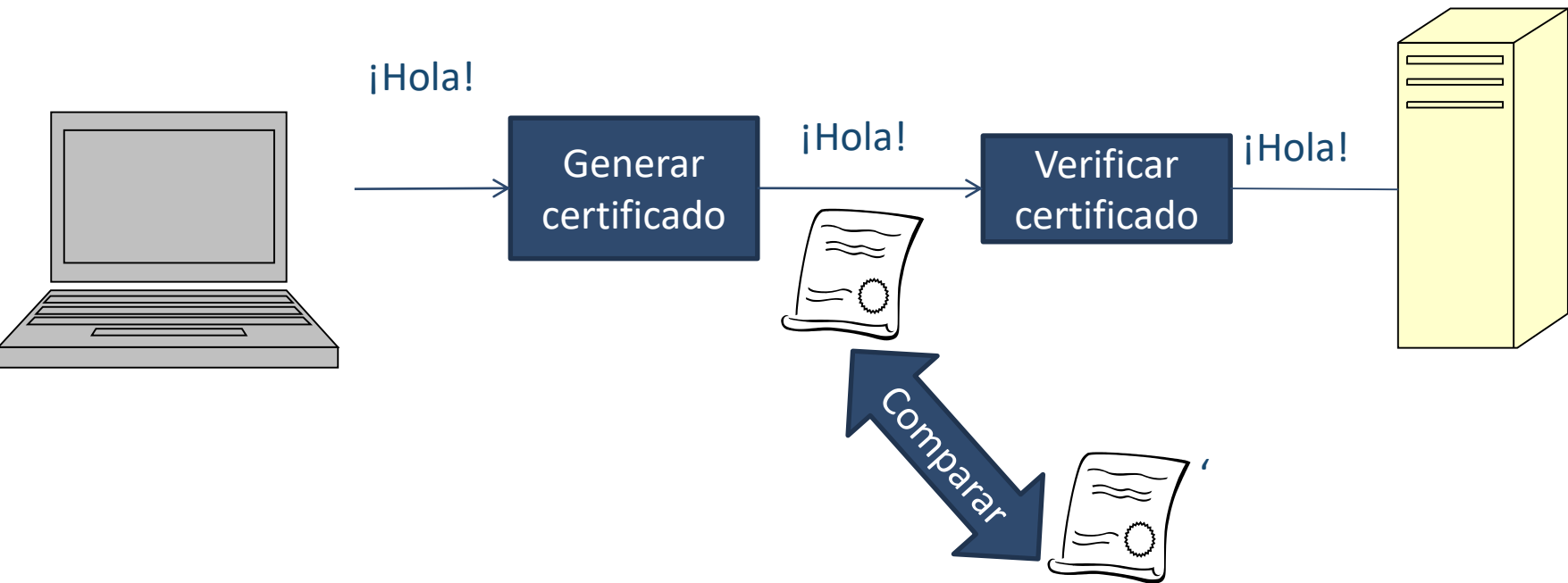
Llave
pública

Llave
privada

Llave pública se genera y se envía cada
cierto tiempo

Llave privada se genera cada cierto
tiempo

Certificados digitales



Tácticas contra amenazas (2/2)

Filtrar entradas del usuario

Injection

Autenticación

Broken authentication

Autorización

Broken access control

Filtrar entradas del usuario

- Ejemplo de una entrada de usuario que borra una tabla de la base de datos

```
String sql = "SELECT * FROM Product WHERE name =" +  
pname + "";
```

```
stmt = conn.createStatement();
```

```
rs = stmt.executeQuery(sql);
```

```
'; DROP TABLE Customers --
```

```
SELECT * FROM Product WHERE name=""; DROP TABLE  
Customers --
```

Filtrar entradas del usuario

- Al filtrar las entradas de usuario se evita que el software sea modificado
- Ejemplo de cómo escribir una consulta evitando inyección de código maligno

```
String selectStatement = "SELECT * FROM Product WHERE name  
= ? ";
```

```
PreparedStatement prepStmt =  
con.prepareStatement(selectStatement);  
prepStmt.setString(1, pname);
```

```
ResultSet rs = prepStmt.executeQuery();
```

Autenticación y autorización: definiciones

- La autenticación consiste en verificar si un usuario puede acceder a una aplicación, dadas unas credenciales. No importa su rol
- La autorización consiste en verificar a qué componentes (o funcionalidades) de una aplicación puede acceder un usuario, dado su rol

Autenticación y autorización: conceptos relacionados

- **Realms o dominio de seguridad:** Es un repositorio de usuarios y grupos para validar los usuarios de una aplicación y son controlados por la misma política de autenticación.
- **Usuario (Principal):** Es un individuo con una entidad registrado en el servidor de aplicaciones. Pueden ser asociados a un grupo
- **Grupo:** Es un conjunto de usuarios con rasgos en común, p.e: grupo de construcción de sw.
 - Puede tener asociados varios roles al igual que un usuario
 - Facilita la administración de usuarios
 - Está relacionado con todo el servidor de aplicación
- **Rol:** Determina el permiso para acceder a un conjunto particular de recursos de una aplicación
 - Relacionado con una aplicación del servidor de aplicación. Ej: administrador, empleado, estudiante
- **Credencial:** Contiene la información usados para autenticarse

¿Cómo se implementan las tácticas vistas con Auth0 y JWT?

Auth0 y JWT (1/2)

- Auth0
 - Auth0 es un servicio externo (cloud) destinado a la gestión de usuarios, credenciales, grupos, roles, etc.
 - Su realm o dominio es una base de datos no relacional
- JSON Web Tokens (JWT)
 - Los JWT son tokens que se agregan a las peticiones Web para garantizar que la fuente de un estímulo es un usuario válido

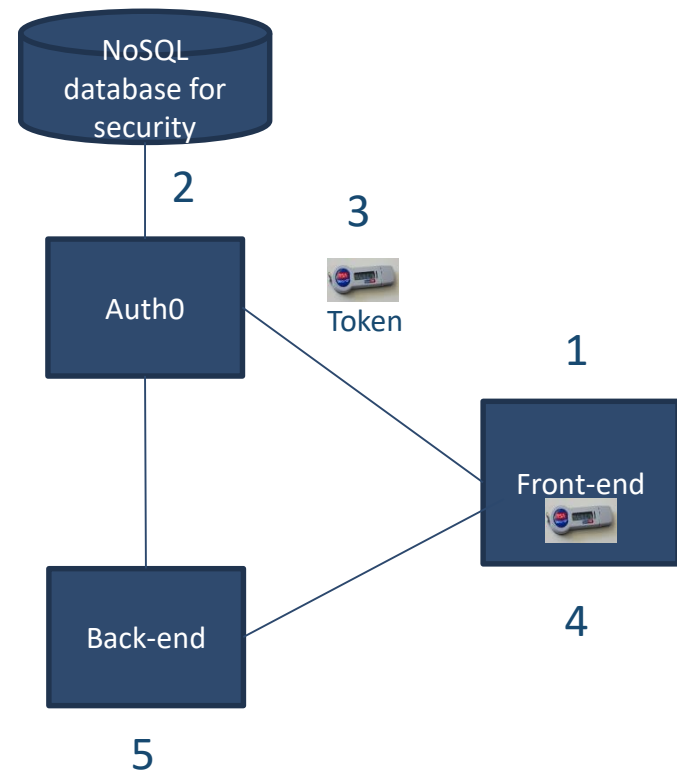
Auth0 y JWT

- **JSON Web Tokens (JWT)**
 - Un JWT se compone de una cabecera, un cuerpo (payload) y un hash
 - La **cabecera** indica el algoritmo con el que el token fue encriptado
 - El **cuerpo** contiene los datos de seguridad, p.e., nombre de usuario, rol
 - El **hash** se usa para mitigar una modificación del token
- A continuación, veremos el funcionamiento de Auth0 y JWT en el contexto de una aplicación donde el front-end está desacoplado del back-end



Funcionamiento de autenticación y autorización con Auth0 y JWT

1. Un usuario hace login en el front-end
2. Auth0 valida las credenciales enviadas desde el front-end. Consulta su base de datos
3. Auth0 emite un token que tiene una expiración
4. Las próximas veces que el front-end envíe una petición, el token es embebido dentro de la petición
5. El back-end recibe la petición, verifica el token y procesa la petición solo si el token es válido



Referencias

- [Bass2013] Bass, L. Clements, P., Kazman, R., “Software Architecture in Practice”, Addison-Wesley, Third Edition, 2013. Capítulo 5
- [Auth0] <https://auth0.com/>